Waybill — a courier routing engine whose report cannot lie

The assignment says TSP, Greedy, and Branch & Bound. Couriers do not solve TSP. This is the plan for building what they actually solve — and for making it structurally impossible for a number in the write-up to disagree with the code that produced it.

BUILT	TESTS	VERIFIED	ROAD DATA	PHASES
Phases 0–1 ✓	186 passing	5 TSPLIB optima	live OSRM	9, gated

What changed in revision 3

Two things happened since revision 2: Phases 0 and 1 were built and measured, and a parallel implementation was reviewed. Both changed the plan.

LEARNED FROM BUILDING

- **The claim contract works.** `SolveReport` refusing to construct is better than a convention. Nine tests now exercise those refusals.
- **Real roads are more asymmetric than expected.** On five Monaco stops, *all 10* pairs differ by direction — largest gap 130 s against 296 s. The 2-opt guard is not hypothetical.
- **A validation gate caught a real bug** in the solver meant to be ground truth. The bottom of the chain needs its own independent reference.

LEARNED FROM THE REVIEW

- **Fabricated data is the top risk**, above every algorithmic concern. A parallel build had 17 of 52 coordinates wrong in one instance and the wrong distance model entirely in another.
- **A hand-written report drifts from its engine.** That build claimed a 96.4% pruning ratio; its own code returned 89.2%.
- **Volume is not evidence.** 7,246 lines with no tests proves nothing.

NEW IN THIS REVISION

- **The evidence bundle (§04).** The report and the demo are *generated* from measured runs, so neither can state a number the engine did not produce.
- **Data integrity rules (§05).** No numeric instance data may be typed into source. Ever.
- **A named list of claims this project will not make (§13).**

00 In plain terms KEPT

No jargon. If you can explain the project this way, you can defend it in a viva.

What is the actual job?

A delivery company has a warehouse and 150 parcels to deliver today. It has 6 vans. Somebody has to decide two things: **which van takes which parcels**, and **what order each driver visits their stops in**.

Get it right and every driver is home by 5pm having burned less fuel. Get it wrong and one van is overloaded, another is half empty, and somebody is still driving at 8pm.

Why can't a computer just try every option?

Because the number of options doesn't grow steadily — it explodes.

10 STOPS	15 STOPS	20 STOPS
181,440	**43.6 billion**	**60.8 quadrillion**
possible routes. A laptop checks all of them before you blink.	possible routes. Now about 44 seconds of solid computing.	possible routes. Checking a billion every second: **1.9 years**.

Twenty stops is a light morning for one driver, so "check everything" stops working almost immediately — and no faster computer rescues you.

So what do you do instead?

The quick way — be greedy. Repeatedly drive to the nearest stop you haven't visited. Like being at a party and always talking to whoever is closest. Instant, reasonable, and because you never planned ahead you end up stranded across the room facing one long drive home.

The careful way — be clever about what you skip. You've found a route costing 87 km. You look at a huge group of routes that all start with the same bad move, and you prove that *even at their luckiest* they cost at least 94. You discard the whole group unopened. That is **Branch & Bound**.

What makes this project different?

Most projects implement one method and print a route. Nobody can tell whether it's good, because there's nothing to compare against.

This one builds both kinds, runs them on identical problems, measures the difference, and **labels every number with how much it is entitled to claim**. "Proven best" and "best we found" are different statements, and the system refuses to confuse them.

It goes one step further, and that step is the point of this revision: **the write-up is generated from the measurements**. Nobody types a result into a document. If a number appears in the report, a command produced it, and that command can be re-run.

Why does that last part matter so much?

Because the most common way these projects go wrong is not a bad algorithm — it is a good-looking number that nothing produced. A report says one thing, the code says another, and nobody notices until someone asks.

A reviewed implementation of this same brief claimed a "96.4% pruning ratio." Its own code, run directly, returned 89.2%. Neither number was checkable, because the report was written by hand.

THE SIX WORDS WORTH KNOWING

TSP

Travelling Salesman Problem. One vehicle, a list of stops: shortest round trip visiting each once.

GREEDY

Any method that always takes the choice that looks best right now. Fast, decent, never guaranteed.

BRANCH & BOUND

Split possibilities into groups, compute each group's best conceivable outcome, discard whole groups whose best case loses to an answer you already hold.

LOWER BOUND

A number you can prove no route in a group beats. If it is ever wrong in the optimistic direction, the whole proof collapses silently.

OPTIMALITY GAP

How much worse a fast answer is than a *proven* best one. Without a proof you don't have this number — you have something weaker.

CVRPTW

The realistic version: multiple vehicles, capacity limits, delivery time windows.

---

01 The problem the title gets wrong KEPT

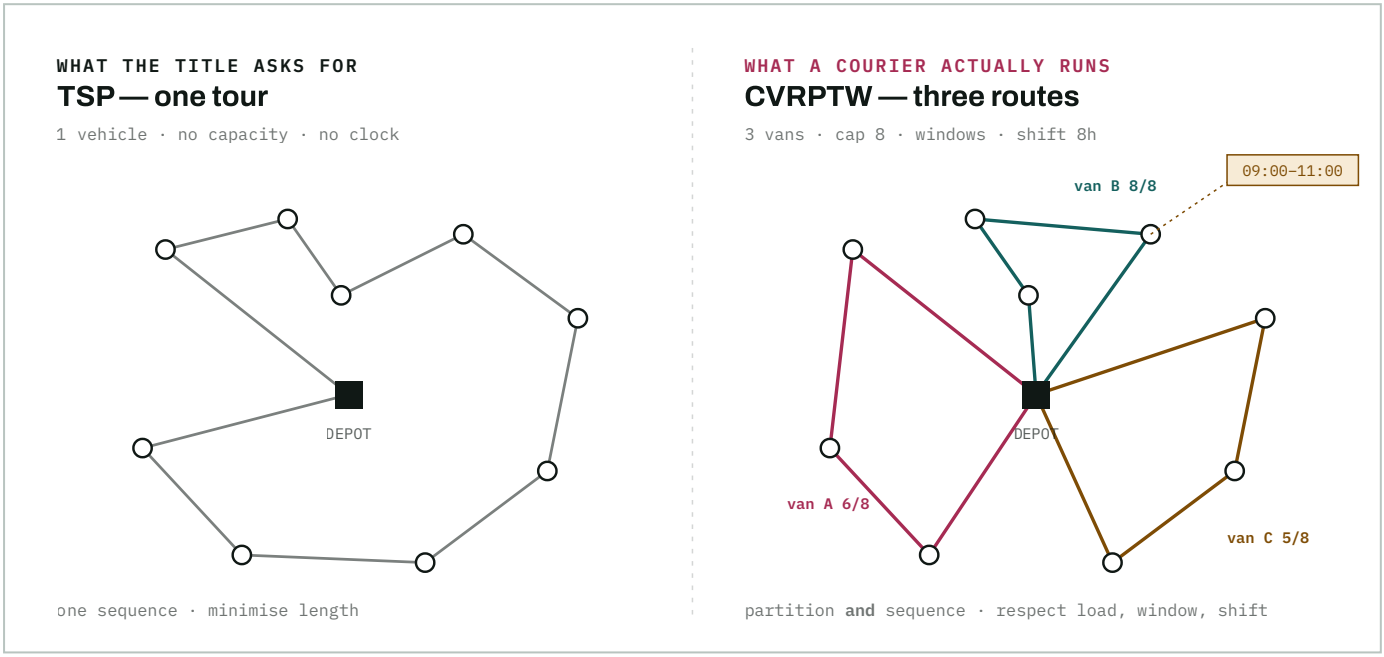
> TSP is the kernel. It is not the system. Building the kernel and calling it a courier system is the single reason these projects read as coursework.

The **Travelling Salesman Problem** asks for the minimum-cost Hamiltonian cycle over `n` nodes. It is NP-hard; a symmetric instance has `(n-1)!/2` distinct tours. A courier fleet violates every simplifying assumption it makes:

- **Multiple vehicles.** Deciding which van takes which stops is a partitioning problem on top of the sequencing problem — and it dominates the cost.
- **Capacity.** A tour optimal in distance is worthless if it needs 1.4 vans.
- **Time windows.** "Between 09:00 and 11:00" turns ordering into scheduling.
- **Service time.** Minutes at the door, often exceeding driving time between adjacent urban stops.

- **Asymmetry.** `d(A,B) ≠ d(B,A)`. Measured, not assumed — see §08.
- **Shift limits.** Legal maximum hours and mandated breaks.

The correct name is CVRPTW. TSP is the special case with one vehicle, infinite capacity, and no clock — and in this codebase that is literally true: one `Instance` type covers all three, and `problem_type` classifies an instance by inspecting it rather than by trusting a label.



Identical nine stops, two different problems. The left optimises a sequence. The right must first decide which van gets which stops, then sequence each, then check every arrival against a clock and every load against a limit. The partitioning decision is where most of the cost — and all of the difficulty — lives.

---

02 Where the project stands NEW

Revisions 1 and 2 were plans. This section reports measurements.

Phases 0 and 1 are built and their gates passed. 2,242 lines of implementation, 1,002 lines of tests, 186 tests passing. Everything below is output from a command in the repository, not an expectation.

Distance conventions reproduce published optima

Our own implementation scoring the published `.opt.tour` files — two independent facts that must agree:

INSTANCE	N	COMPUTED	PUBLISHED	MATCH
eil51	51	426	426	✓
berlin52	52	7542	7542	✓
st70	70	675	675	✓
pr76	76	108159	108159	✓
kroA100	100	21282	21282	✓

Real roads are asymmetric, measured

Five real Monaco locations through a self-hosted OSRM against a pinned OpenStreetMap extract:

PROVIDER	SYMMETRIC	MATRIX HASH	ASYMMETRIC PAIRS
haversine	true	587a429c5c88b7d6	0 of 10
osrm (driving)	**false**	b04816a09b5f83ce	**10 of 10**

Largest directional gap: **130 seconds** — 296 s one way against 426 s back, a 44% difference from one-way streets. Every pair differed. This is the concrete justification for the operator constraints in §08, and it is stronger than anticipated: revision 2 argued asymmetry *matters*; the measurement says it is the normal case, not an edge case.

> **A GATE CAUGHT A REAL BUG — IN THE GROUND TRUTH ITSELF**
>
> The brute-force solver initially applied an invalid symmetry reduction: it fixed which customer was visited first, which discards genuine tours rather than duplicates. On a three-customer instance it enumerated 2 of 3 and could return a non-optimal answer stamped `PROVEN_OPTIMAL_TSP`.
>
> It was caught by comparing against an enumeration applying *no* reduction at all. The lesson generalises: **the bottom of a validation chain needs its own independent reference**, because everything above inherits its errors. That reference test is now permanent.

03 The claim model IMPLEMENTED

Revision 2 specified this. It is now code, and the code refuses.

Exactness is a property of one formulation, one instance, and one solver run — never of the application. Every solve returns an explicit status, and nothing may be worded more strongly than its status permits.

STATUS	MEANS	PERMITTED WORDING
`PROVEN_OPTIMAL_TSP`	Exact solver finished with a valid proof condition on a single-vehicle instance.	"Optimal" — that instance only.
`EXACT_TIMEOUT_WITH_INCUMBENT`	Exact search hit its budget before proving optimality.	Incumbent *and* lower bound. Never "optimal".
`HEURISTIC_FEASIBLE_CVRPTW`	Multi-vehicle plan passed the independent feasibility gate.	"Feasible." Quality relative to a named comparator.
`HEURISTIC_INFEASIBLE`	Violates capacity, window, shift, or visit-once.	Reject. Never surfaced as a route.
`REFERENCE_GAP_ONLY`	Compared to OR-Tools or a best-known value.	"Gap to reference" — never "optimality gap".

Enforcement, not convention

The difference between a documented rule and an enforced one is the difference between a project that holds and one that drifts. `SolveReport` **cannot be constructed** if it:

- claims proven optimality with any `stop_reason` other than `OPTIMAL`;
- claims TSP optimality for a CVRPTW run — a single-vehicle certificate says nothing about a fleet plan;
- reports a timeout without the lower bound that makes the residual gap knowable;
- supplies a reference value without provenance naming the comparator;
- carries a lower bound above its own objective, which would mean the bound is invalid and may already have pruned the optimum.

A `headline()` method returns the strongest permitted phrase, so display code never composes its own wording. The rules live in one place and are tested by nine cases that each assert a refusal.

---

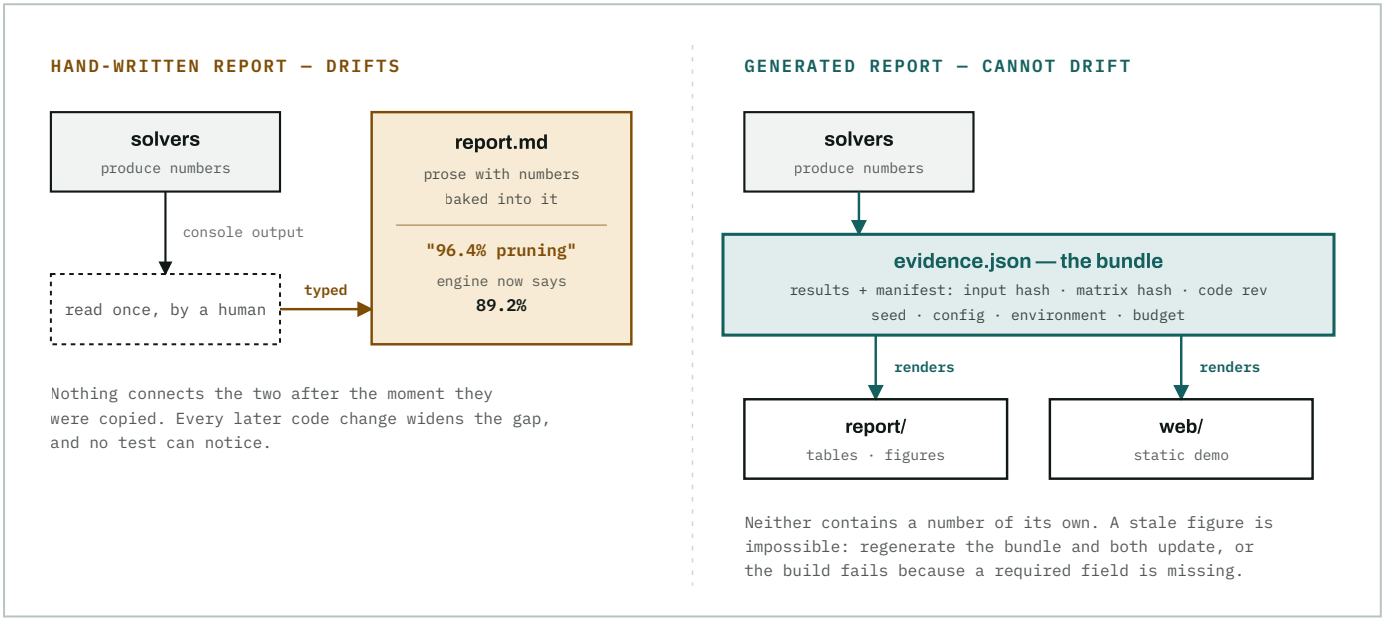
04 Architecture — and the evidence bundle

NEW · THE CENTREPIECE

> The most common failure in a project like this is not a wrong algorithm. It is a right-looking number that nothing produced. The fix is architectural: make the report a build artefact rather than a document.

A reviewed implementation of this same brief stated a 96.4% pruning ratio in its report while its own engine returned 89.2%. Neither figure was wrong on purpose. The report was simply written by hand at one moment and the code changed at another, and nothing connected them. That failure mode is available to any project whose write-up is prose.

So `bench/` emits a single **evidence bundle**: a JSON file containing every measured result together with the manifest that produced it — input hashes, matrix hash, code revision, solver configuration, seed, environment, budget. **The report and the demo both render from that bundle and from nothing else.** Neither can state a number the engine did not produce, because neither contains any numbers of its own.



The difference is one file. On the left a human is the only link between engine and report, so the two drift the moment either changes. On the right the bundle is the only source of numbers for both outputs, so "the report says something the code doesn't" stops being a thing that can happen. The manifest travels with the results, which is what lets a reader trace any figure back to the run that produced it.

The demo runs without a server

A second benefit falls out of the bundle, and it matters more than it looks. Because the demo renders from a static JSON file, **the whole thing opens from disk** — no Python process, no Docker, no build step for an examiner who wants to look at it. Live interactive solving, where a marker is dragged and routes re-optimise, is an *additional* mode behind the FastAPI service, not a prerequisite for seeing the evidence.

That inverts the usual fragility. The demo degrades gracefully: with the server it is interactive, without it, it still shows every measured result.

Layout

```
waybill/
├── core/                  # stdlib only — enforced by tools/check_purity.py  built
│   ├── model/ distance/ construct/ exact/ local/ meta/
│   ├── constraints/ eval/ reporting/
├── adapters/              # every socket, file, and DB call  built
│   ├── distance/            # haversine, osrm_client, sqlite cache
│   └── storage/
├── bench/                 # experiment runner → the evidence bundle
│   ├── tsplib/              # loader + published optima  built
│   ├── experiments/         # one module per experiment family (§10)
│   ├── manifest.py          # provenance record attached to every result
│   └── bundle.py            # emits evidence.json  ← single source of numbers
├── report/                # renders FROM the bundle. contains no numbers.
├── web/                   # static demo, reads evidence.json; live mode optional
├── api/                   # thin FastAPI — only for interactive mode
├── infra/                 # docker compose: OSRM + pinned OSM extract  built
├── data/                  # external inputs, checksummed, never hand-typed  built
└── tools/                 # check_purity, verify_gate  built
```

> **THE RULE THAT MAKES IT MECHANICAL**
>
> A build step greps `report/` and `web/` for numeric literals outside of layout constants. Any bare figure that is not read from the bundle fails the build. It is a crude check, and it is exactly strong enough to prevent the failure it targets.

---

05 Data integrity NEW · HIGHEST RISK

> The review found that the single largest threat to this project is not an algorithm. It is a plausible number in the wrong place.

A parallel implementation of this brief inlined benchmark coordinates directly into its source. Checked against the real files: one instance had **17 of 52 coordinates wrong** — correct for the first 35, then drifting, with some values recycled from other positions and the real file's last two absent entirely. Scoring the genuine published optimal tour against that data gives **16,883 instead of 7,542**.

A second instance was worse in a subtler way. `bayg29` is `EDGE_WEIGHT_TYPE: EXPLICIT` — its true distances live in a matrix inside the file, and its coordinates are `DISPLAY_DATA`, present only for drawing. Applying a Euclidean formula to display coordinates produces a different problem, which was then paired with the real problem's published optimum. Summed across all pairs the two models differ by **6.07×**.

That instance's own benchmark output reported a nearest-neighbour gap of **621%**. Nearest neighbour runs about 25% above optimal. The number was visible in the output the whole time.

Rules

- **No numeric instance data may be typed into source. Ever.** Coordinates, distance matrices, and published optima are loaded from files in `data/`. A model reproducing a table from memory is confabulation with a plausible failure mode, and this is the only defence that reliably works.
- **Every input file carries a recorded SHA-256**, checked on load. A silently edited fixture is then impossible rather than merely unlikely.
- **Provenance is recorded per file** — source URL, retrieval date, and the archive it came from — so any reader can re-fetch and compare.
- **Every published optimum is cross-checked** by scoring the real `.opt.tour` with our own implementation. Two independent facts agreeing; the test prints both when they do not, so the cause is immediately visible.
- **Support `EXPLICIT` before using an `EXPLICIT` instance.** Many classic TSPLIB instances are matrix-defined. Treating their display coordinates as geometry is silently wrong.
- **A gap above 100% is treated as a data fault, not a result**, and fails the build. No heuristic in the ladder is capable of being twice as bad as optimal; if one appears to be, the instance is wrong.

> WHY THIS OUTRANKS THE ALGORITHMS
>
> An algorithmic bug produces a number that is wrong. A data bug produces a number that is wrong *and internally consistent* — every downstream comparison agrees with every other, the plots look reasonable, and nothing fails. The only thing that catches it is an external reference the project does not control.

---

06 Solver scope REVISED

Six mandatory methods, fully validated, beat nine partially tested ones.

METHOD	CLASS	TIER	PURPOSE
Brute force	EXACT	BUILT	Ground truth to n≈10; root of the validation chain
Held–Karp DP	EXACT	MANDATORY	Exact reference to the memory wall; exposes that wall as a result
B&B + reduced-cost bound	EXACT	MANDATORY	The assignment's named technique; bound-comparison baseline
B&B + Hungarian bound	EXACT	MANDATORY	Second bound — without it there is no bound experiment
B&B + 1-tree / subgradient	EXACT	RECOMMENDED	A structurally different relaxation; gated on the validity invariant
Nearest neighbour	HEUR	MANDATORY	The assignment's "Greedy"; the baseline to beat
Clarke–Wright savings	HEUR	MANDATORY	Fleet-aware construction; partitions and sequences at once
Or-opt / relocate	HEUR	MANDATORY	Asymmetry-safe local search (§08)
2-opt (symmetry-guarded)	HEUR	MANDATORY	Classic improvement, with the guard that makes it honest
Simulated annealing	HEUR	RECOMMENDED	One well-instrumented metaheuristic, for the anytime curve
OR-Tools	RULER	REFERENCE	External baseline in `bench/` only, never inside a solver

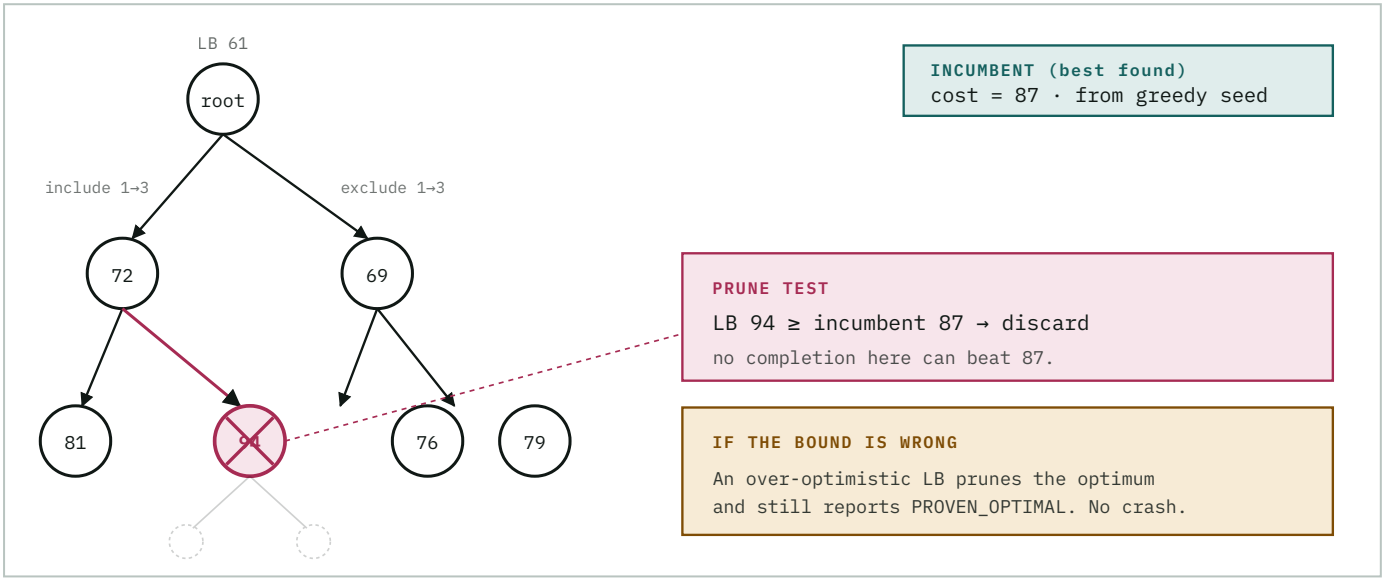
> **A METAHEURISTIC THAT DOES NOTHING STILL RETURNS A NUMBER**
>
> In the reviewed implementation, simulated annealing and Clarke–Wright returned *identical* costs on eil51 — 458 for both. The annealer improved on its starting solution zero times, and reported a result anyway.
>
> An under-calibrated SA is indistinguishable from a working one unless you look. So SA must record **moves proposed, moves accepted, accepted-worsening moves, and best-cost-over-time**, and a run where acceptance is zero fails the phase gate rather than producing a data point. A cooling schedule that starts too cold accepts nothing and degenerates into a slower local search; one that starts too hot spends its budget as a random walk. The instrumentation is what tells the two apart, and the anytime curve is meaningless without it.

Choose SA over ALNS. SA is roughly 80 lines with three parameters and its acceptance behaviour follows directly from the cooling schedule, so it can be derived and defended on a whiteboard. ALNS is several hundred lines of destroy/repair operators with adaptive weights — more impressive when it works, much harder to validate and to explain under questioning. If SA is finished, instrumented, and its curve generated with time to spare, ALNS becomes a genuine stretch goal.

07 Branch & Bound — the experimental design KEPT

> Branch & Bound is a bound with a search wrapped around it. How far it scales is the experiment's output, never its premise.

Maintain the best complete tour found (the **incumbent**); for each partial tour compute a **lower bound** on the best possible completion; if that bound is already ≥ the incumbent, discard the subtree unexplored.



One prune eliminates a subtree — one invalid bound eliminates the answer. The amber panel is the failure this project guards hardest against: a bound that is too high does not error, it discards the branch containing the optimum and returns a confident wrong answer.

Control variables

The bound experiment means something only if the bound is the *only* thing that changes. Hold constant: the instance, the incumbent initialisation (the same greedy-plus-local-search seed tour for every variant), the branching rule, the search strategy, and the timeout.

- **Record bound computation time separately.** A tighter bound that costs too much per node can lose overall — visible only if the two costs are measured apart.
- **Record peak frontier size**, so best-first's memory growth is a curve rather than a crash.
- **On timeout, save incumbent and best remaining lower bound.** A timeout then yields a certified residual gap instead of a hole in the table.
- **Fix one search strategy as default** for the primary bound comparison, or two things vary at once.

THE SENTENCE TO FILL IN

"On this hardware and instance family, exact method X remained provably optimal through n = ___ under a ___-second budget; beyond that, heuristic Y achieved a median gap of ___% in ___ ms. Changing only the lower bound reduced explored nodes by ___×."

Blanks are filled by the bundle, not by hand. Under the §04 architecture that is not a discipline — it is the only way the sentence can be rendered at all.

---

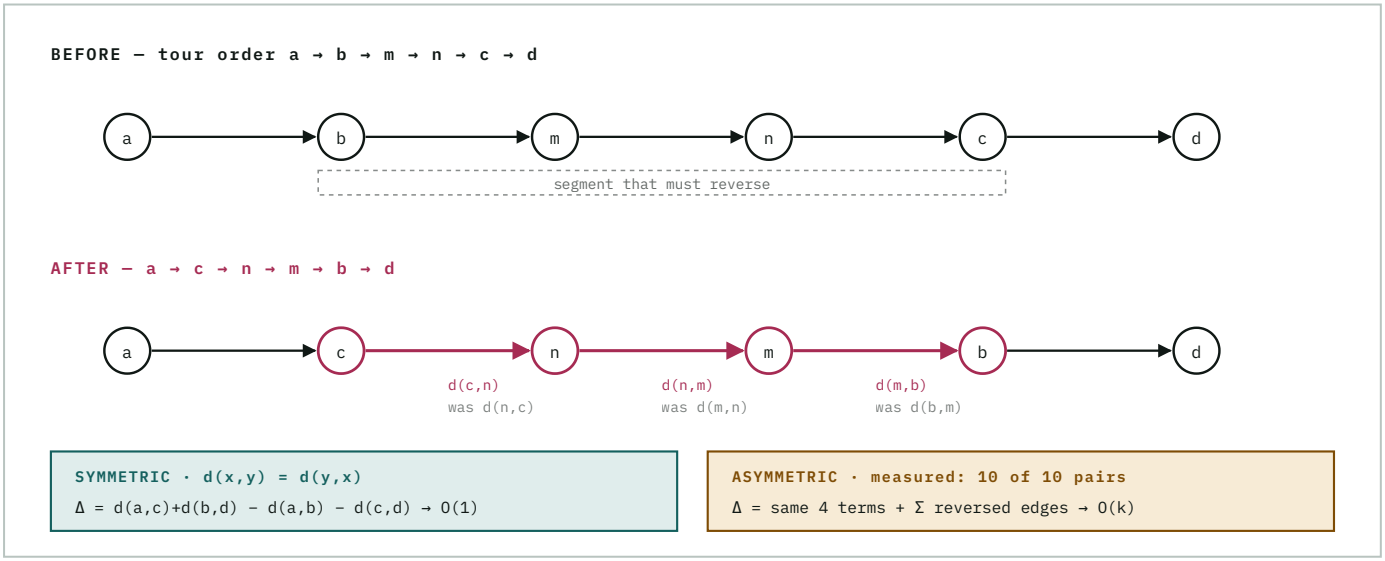
08 The asymmetry trap NOW MEASURED

> Textbook 2-opt is not valid on an asymmetric matrix. Revision 2 argued this; the OSRM measurement settles it.

2-opt removes edges `(a,b)` and `(c,d)`, adds `(a,c)` and `(b,d)`, and to keep the tour connected the path from `b` to `c` must be **traversed in reverse**.

On a symmetric matrix that reversal is free — every internal edge `(x,y)` becomes `(y,x)` at the same cost — so the delta collapses to four terms and evaluates in constant time. That `O(1)` delta is the entire reason 2-opt is fast enough to use.

On an asymmetric matrix every reversed internal edge changes cost. The delta needs a sum over the whole reversed segment: `O(k)`. Keep the four-term formula and the search is optimising **a quantity that is not the tour length** — without erroring.



The reversal is free only when the matrix is symmetric. Under the OSRM matrix measured in §02, *every* pair of the five Monaco stops differed by direction, so every accent edge above changes cost when the segment flips. The four-term delta is simply the wrong number.

What follows

- **Or-opt and relocate are mandatory, not optional.** They move a short segment without reversing it, so their delta stays `O(1)` and correct under asymmetry.
- **2-opt carries a symmetry guard.** `DistanceMatrix.is_symmetric` is computed once at construction and already exists. 2-opt asserts on it or switches to the honest `O(k)` delta — never silently using four terms on asymmetric input.
- **TSP experiments run on symmetric instances** (TSPLIB and random Euclidean), so classical comparisons stay comparable to published results. The **asymmetric road matrix belongs to the CVRPTW demo and the road-matrix experiment.**

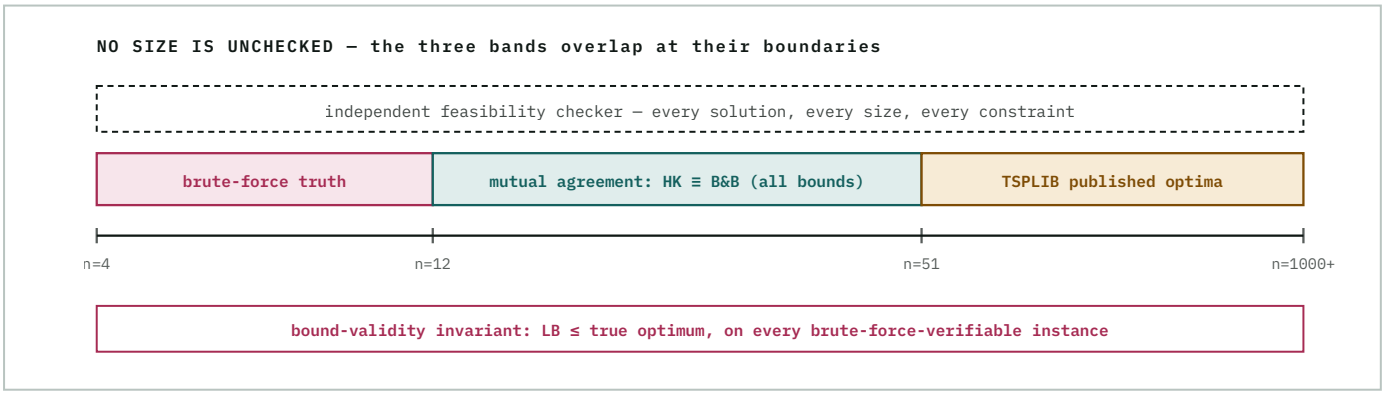
> A FINDING ALREADY HALF-COLLECTED
>
> Solve one instance under the symmetric Haversine matrix, then under the OSRM road matrix over identical stops. Report how tour length, visiting order, and operator validity change. §02 has the matrices and their hashes; what remains is running the solvers across both and recording the difference.

---

09 Validation — an unbroken chain KEPT

Brute force is trustworthy to about `n = 12`. TSPLIB's published optima start at `n = 51`. Between them nothing external checks Held–Karp or B&B — and that band is exactly where the interesting exact-solver results live, so it cannot be a gap.

It closes with mutual agreement: three independently written exact solvers sharing no code path. Held–Karp is a dynamic program over subsets, B&B is a tree search with a bound, brute force is exhaustive enumeration. A common bug producing identical wrong answers across all three is vanishingly unlikely.



Every problem size has something checking it. The bands overlap rather than merely touch, so each handover is itself verified. The two full-width rules apply everywhere: the feasibility gate on all solutions, and the bound-validity invariant protecting the exact solvers from the silent failure in §07.

TIER	CHECK	COVERS	STATE
1	Brute force on random instances	n ≤ 12 — the root of trust	built
2	Brute force ≡ Held-Karp ≡ B&B (every bound)	overlap at 12, then 13–50 by agreement	Phase 2
3	TSPLIB loader reproduces published tour costs exactly	n ≥ 51, external ground truth	built ✓
4	Independent feasibility checker on every output	all sizes, all constraints	built
5	Bound validity: LB ≤ true optimum	every bound, 1,000 instances	Phase 3
6	OR-Tools stored separately with provenance	reference only — never truth	Phase 6
7	End-to-end real-road run: stops → matrix → plan → map	the integration path	Phase 7

10 Evidence discipline REVISED

> **PRIMARY RESEARCH QUESTION**
>
> At what problem size and compute budget does exact optimisation stop being worth its cost — and how much solution quality is given up by switching to scalable heuristics?

Every experiment answers part of this. One question makes the report an argument rather than a catalogue.

Experiment families

- **Exact crossover sweep.** Increase `n` until each exact method hits its time or memory budget; record solved fraction, wall time, peak memory, proof status. Locates the wall — the central result.
- **Bound quality.** Identical instance, incumbent, branching rule and strategy; only the lower-bound function changes. Compare nodes explored and pruned, plus bound computation time.
- **Heuristic gap.** Where the optimum is known, repeated seeded instances; report the gap distribution with uncertainty.
- **Anytime quality.** Large instance; incumbent quality against elapsed time, with acceptance counters proving the annealer is annealing.
- **Road-matrix effect.** Same stops under Haversine and under OSRM; quantify how asymmetry changes tour cost, visiting order, and operator validity.

- **Feasibility engine.** Naive downstream recomputation versus forward-slack acceleration on identical accepted moves.

Statistics that suit the measurement

- **Runtime: median and IQR**, optionally p90. Runtime distributions are right-skewed by garbage collection, cache effects, and OS scheduling; a mean misleads.
- **Quality gap: mean with a bootstrap confidence interval** over ≥30 seeded instances per size. Bootstrap rather than a normal interval, because gap distributions are bounded below at zero and right-skewed, so a symmetric interval can extend into impossible territory.
- **Exact-solver success: solved / attempted** under a fixed budget — never statistics over only the runs that finished, which silently discards the hardest instances and flatters the method.
- **B&B: node counts and pruning ratio** alongside time, since time is contaminated by implementation overhead in a way node counts are not.
- Every figure names CPU, RAM, OS, interpreter version, seed policy, and time budget — carried in the manifest, not typed into a caption.

Invariant tests

Property tests over random instances, on every commit:

- **A lower bound never exceeds the true optimum** on any brute-force-verifiable instance. The most important test in the project — the only thing between you and a confidently wrong "proven optimal".
- Route cost is unchanged by serialising and deserialising an instance.
- Every customer appears exactly once unless explicitly recorded as unserved.
- No accepted local-search move ever worsens feasibility.
- Identical matrix + seed + configuration produces an identical objective and result trace.
- Tour cost is invariant under rotation, and under reversal *only* when the matrix is symmetric — a direct test of §08.
- **No reported gap exceeds 100%** — a data-integrity tripwire (§05), not a quality check.

Failure corpus

When a random test finds a counterexample, persist that instance as a named regression fixture rather than trusting the seed. Seeds stop reproducing the moment the generator changes, and the most valuable tests in optimisation software are the small ugly instances that once broke an invariant — a zero-length edge, a window that opens before the vehicle can physically arrive, two customers at identical coordinates.

11 Build plan

2 OF 9 DONE

Each phase has an exit gate — a falsifiable condition, not a description of intent.

> Effort split: **70% core and benchmark evidence · 20% real-road and CVRPTW integration · 10% interface.** A polished interface cannot compensate for an unvalidated exact solver — and under §04 the interface is largely generated anyway.

PHASE 0 ✓

Contracts and truth COMPLETE

Model layer, TSPLIB conventions, independent feasibility checker, status and `SolveReport` contract, `Solver` protocol, brute force, core purity checker.

> **GATE PASSED**
> Five TSPLIB instances score their published optima exactly; brute force agrees with unreduced enumeration on 60 instances; purity check fails on a planted import.

PHASE 1 ✓

Distance contract and adapters COMPLETE

Haversine, self-hosted OSRM client, SQLite cache — all in `adapters/`. Docker pipeline against a pinned OSM extract.

> **GATE PASSED**
> Same stops run offline and on OSRM; both log a matrix hash; symmetry flag correct for each (10 of 10 pairs asymmetric on road data).

PHASE 2

Assignment-complete solver core

Nearest neighbour, Clarke–Wright savings, Held–Karp DP, and B&B with the reduced-cost bound. Plus TSPLIB `EXPLICIT` support, which widens the benchmark set and removes the trap that produced a 6× error elsewhere. **The assignment's three named techniques are satisfied at the end of this phase.**

> **EXIT GATE**
> All three exact solvers return identical objectives across a broad seeded suite. Any disagreement blocks Phase 3.

PHASE 3

The Branch & Bound experiment

Hungarian bound, then 1-tree with subgradient ascent gated on the validity invariant. Full instrumentation: nodes created/explored/pruned, peak frontier, incumbent trace, bound time recorded separately. DFS and best-first both available, one fixed as default.

> **EXIT GATE**
> Bound comparison renders from the bundle, and the validity invariant passes on 1,000 random instances for every bound.

PHASE 4

Heuristic quality ladder

Or-opt and relocate first (asymmetry-safe), then 2-opt with its symmetry guard, then simulated annealing with acceptance instrumentation. Neighbour lists and don't-look bits — without them local search is unusable past a few hundred stops.

> **EXIT GATE**
> Gap-vs-n and anytime curves render from the bundle with bootstrap intervals, *and* SA records a non-zero accepted-worsening count. An annealer that never accepts a worsening move fails this gate.

PHASE 5

CVRPTW constraints

Capacity in parcel count, hard time windows, service time, shift limits, multi-vehicle throughout. Forward time slack for `O(1)` move feasibility. Every operator becomes constraint-aware.

> **EXIT GATE**
> 150 stops, 6 vehicles, real windows: passes the independent checker and reports `HEURISTIC_FEASIBLE_CVRPTW` — never "optimal".

PHASE 6

The evidence bundle

Sweep runner, manifests, OR-Tools reference column, and `evidence.json`. The report renders entirely from it; the numeric-literal check turns on. **The grade lives here.**

> **EXIT GATE**
> One command regenerates bundle, tables, and figures in a clean environment; every figure traces to a manifest; no numeric literal survives in `report/`.

PHASE 7

Static demo, then live mode

A map interface rendering the bundle from disk — no server needed. Real OSRM polylines, per-stop ETAs, and a provenance panel showing method, formulation, status, elapsed time, units, comparator, and whether the number is proven or reference-relative. *Then* the thin FastAPI service for interactive solving.

> **EXIT GATE**
> The demo opens from disk with no processes running and shows real measured results; live mode adds interactivity without duplicating any algorithm logic.

PHASE 8

Stretch only

Dynamic insertion of new orders, failed-delivery handling, frozen completed stops. Optionally ALNS compared against SA.

> **EXIT GATE**
> Attempted only if Phase 6 is complete and stable.

Stop rules

- Do not add a solver while an existing exact solver disagrees with brute force.
- Do not start interface polish before the bundle regenerates from a clean run.
- Do not implement Phase 8 while any report figure is still hand-written.
- Do not optimise for speed before the counters that explain where time goes exist.
- Do not compare solvers that saw different matrices — the matrix hash makes this checkable.
- Do not accept any measured gap above 100% as a result; treat it as a data fault (§05).

---

12 Definition of done REVISED

Every item verifiable by someone else, on their machine, from the repository.

AREA	ACCEPTANCE TEST
Exact correctness	Every exact solver agrees with brute force across a large seeded corpus; unsupported cases explicitly excluded.
Validation continuity	No size band unchecked: brute force, mutual agreement, and TSPLIB overlap at their boundaries.
Bound validity	LB ≤ optimum passes for every bound on 1,000 random instances.
Data integrity	Every input file loads from disk against a recorded checksum; no numeric instance data appears in source.
Published validation	At least five named TSPLIB instances scored under the correct convention against published optima.
B&B evidence	At least two bounds compared on nodes explored, pruned, bound cost, runtime, and peak frontier.
Crossover result	The practical exact-method size wall reported as measured, on named hardware, under a stated budget.
Heuristic evidence	Gap-vs-n from ≥30 seeded instances per size with bootstrap intervals.
Metaheuristic honesty	SA reports non-zero accepted-worsening moves and a monotone best-cost trace.
Asymmetry handling	Road-matrix experiment reported; 2-opt's symmetry guard demonstrated to trigger.
Real roads	At least one full run on a self-hosted OSRM asymmetric matrix with real route geometry.
CVRPTW feasibility	Realistic fleet instance passes the independent checker; status "feasible", never "optimal".
Reproducibility	One command regenerates the bundle, tables, and plots from pinned inputs and seeds.
Report integrity	**No numeric literal in `report/` or `web/`.** Every figure renders from the bundle.
Claim discipline	Every displayed gap names its comparator; every "optimal" traces to a valid proof.
Report quality	Findings include at least one result that contradicted an initial expectation, and explain why.

13 What this project will not claim NEW

> Naming the limits in advance is what makes the remaining claims credible. Every line below is a sentence that will not appear, however tempting it becomes near a deadline.

- **Not "our solver is fast."** A Python implementation is not competitive with LKH or OR-Tools and will not pretend to be. It is instrumented, validated, and explainable — different goals, honestly stated.
- **Not "the fleet plan is optimal."** No CVRPTW result is ever proven optimal here. Those plans are feasible, and their quality is stated relative to a named comparator.

- **Not "X% better" without naming the baseline**, the instance, the budget, and the hardware.
- **Not "real-time."** Nothing is deployed, load-tested, or run against live traffic. The routing data is a pinned extract.
- **Not "production-ready."** It is a measured study with a working system around it.
- **Not a result that was not measured on this machine.** Literature values may motivate an experiment; they never fill a blank.

The report as an argument

1. **Problem mismatch.** TSP is the kernel; courier routing needs partitioning, capacity, time, and asymmetry.
2. **Truth source.** Exact methods establish what "best" means where proof is possible — with an unbroken chain showing no size went unchecked.
3. **The crossover.** Exact methods become impractical; here is exactly where, on this hardware, under this budget.
4. **The quality ladder.** Constructive heuristics are nearly free; local search buys most of the remaining quality; metaheuristics trade rising time for shrinking gains.
5. **Bound engineering.** Tighter bounds eliminate orders of magnitude of search — and their own cost is part of the trade.
6. **Reality check.** Real asymmetric matrices change what a valid route is and invalidate a textbook operator. Here is the measurement.
7. **Reproducibility.** Every number is rerunnable and every claim carries its proof or reference status.

> THE STANDARD TO HOLD
>
> The strongest version of this project is not the one with the most algorithms or the prettiest map. It is the one where a sceptical examiner can pick any number in the report, ask "how do you know that," and receive an answer that ends in a command they can run.
